AutoBench Service — 12 Parameterized Runs (cross-cluster)

Topology: AutoBench Service on **ykt3**, benchmark workloads on **ykt2** (cross-cluster split). **LLM:** shared RH LiteLLM base `https://litemaas.rhoai.rh-aiservices-bu.com/v1`, default model `openai/Qwen3.6-35B-A3B`. **Auth:** `benchmarker` ROPC token (realm `rossoctl`), same user configured on both ykt2 and ykt3 Keycloaks. **Date:** 2026-08-10/11. All runs invoked through the Service HTTP API (deploy → run → report → S3 export).

The canonical 12 runs are the upstream harness's `deploy-and-evaluate` comparison (`./deploy-and-evaluate.sh --agent tool_calling`).

> **Update (2026-08-28): the layer-3 `422`s below are superseded.** Runs #5/#6/#8 were rejected with a `422` because, at the time, plugin-preset composition wasn't enactable over HTTP. That is now implemented end-to-end via **Option B** — `plugin_preset` / `plugins` / `on_error` flow Service → backend `AgentRuntime.spec` → operator webhook, which renders the per-agent `authbridge-config-<agent>` pipeline. The status cells below preserve the original snapshot; e2e re-runs of #5/#6/#8 under the custom preset-enabled images are tracked in Phase 4.

Summary table

#	Run	Status	Result / reason
1	gsm8k — baseline (1 task)	✅ e2e	**1/1 = 1.00**, 7s
2	gsm8k `--max-tasks 10`	✅ e2e	**7/10 = 0.70**, 79s
3	gsm8k `--max-tasks 50 --max-parallel-sessions 4`	✅ e2e	**36/50 = 0.72**, 135s
4	gsm8k `--model openai/Azure/gpt-4o-mini 50t/4p`	⚠️ path-validated, no passing run	Agent redeployed with swapped model (`LLM_MODEL` confirmed), run executes but **0/5** — only alt catalog model (`Qwen2.5-VL-7B`) isn't tool-calling capable; canonical `gpt-4o-mini` absent. Catalog limit, not Service.
5	gsm8k `--plugin-preset auth-only 50t/4p`	❌ not enactable	**422** — layer-3 plugin pipeline requires per-agent `authbridge-config-<agent>` ConfigMap overlay (kubectl) the HTTP-only Service can't do
6	gsm8k `--plugin-preset ibac-only 50t/4p`	❌ not enactable	**422** (same reason)
7	gsm8k `--plugin-preset full 50t/4p`	⚠️ partial	`authbridge_enabled=true` injects the sidecar (`agent` + `authbridge-proxy`), **3/5 = 0.60**, 37s. Cluster-default pipeline; exact `full` composition not guaranteed.
8	gsm8k `--plugin-preset full --plugin ibac:observe 50t/4p`	❌ not enactable	**422** (layer-3 observe selection)
9	tau2 `--max-tasks 10` (multi-turn)	❌ failed (timeout)	Run exceeded wall timeout; even a fresh-MCP **1-task** and **3-task** run hang. tau2 MCP instability + no per-call timeout.

#	Run	Status	Result / reason
10	tau2 `--max-tasks 20 --max-parallel-sessions 4`	❌ failed (timeout)	Same failure mode as #9
11	appworld `--model gemini-2.5-pro --max-tasks 5`	❌ blocked externally	appworld MCP image `ghcr.io/exgentic/exgentic-mcp-appworld:latest` is arm64 → `Exec format error` on ykt2 amd64 nodes
12	appworld `--model gemini-2.5-pro 20t/4p`	❌ blocked externally	Same as #11

Tally: 4 pass e2e with real numbers (#1, 2, 3, 7) · 1 path-validated (#4) · 3 rejected-by-design 422 (#5, 6, 8) · 2 fail on tau2 MCP infra (#9, 10) · 2 externally blocked (#11, 12).

> **Update (2026-09-07): the appworld block on #11/#12 above is superseded — appworld runs e2e on both platforms.** Two separate causes were resolved. First, the arm64-only manifest: upstream `ghcr.io/exgentic/exgentic-mcp-appworld:latest` now publishes a genuine `linux/amd64` + `linux/arm64` index, so there is no `Exec format error` on ykt2's amd64 nodes. Second — and this was the deeper one — the original blocker was the **wrong MCP surface** rather than the architecture: the image must come from the exgentic `exgentic_benchmarks` framework and expose the `list_tasks` / `create_session` / `evaluate_session` / `delete_session` orchestration contract (verified live: 168 tasks). On the v1.26 12-run matrices both legs reach terminal `succeeded` with bounded per-task verdicts on **both** OpenShift and KinD. `pass_rate` is **0.0 by design** — gemini-2.5-pro driving the generic `tool_calling` agent solves no appworld tasks; the acceptance gate is that the pipeline runs tasks to completion and scores them honestly, which it does. Also note the tau2 failures in #9/#10 above are long since fixed (per-task timeouts + a 4Gi MCP): those legs now score 0.75-0.9. Everything in this file is the 2026-08-10/11 snapshot; see `SERVICE_DESIGN_DECISIONS.md` and the generated `results/12run-report-v1.26-*.md` for current numbers.

Elaboration

#1–3 (gsm8k) — full path verified. ykt3 Service → ykt2 agent + MCP over `apps.ykt2` routes → RH LiteLLM → S3 export (`run.json` lands in bucket `rossoctl-benchmarking`, prefix `ykt3-to-ykt2/`). Pass rates (0.70–0.72) are within model/sampling variance of the earlier ykt3-only runs; not a regression.

#4 (model swap) — mechanism works, backend has no suitable model. The Service correctly accepts `--model`, redeploys a distinct agent with that model baked in (`LLM_MODEL` / `EXGENTIC_SET_AGENT_MODEL` confirmed on the pod), and runs the tasks — so the swap plumbing is proven end-to-end. What we can't produce is a *passing* swapped run: this LiteLLM exposes only 4 models (`Granite-Vision-3.2`, `Nomic-embed-text-v2-moe`, `Qwen2.5-VL-7B-Instruct`, `Qwen3.6-35B-A3B`) and only the default `Qwen3.6-35B-A3B` supports tool-calling; `gpt-4o-mini` isn't provisioned and `Qwen2.5-VL-7B` fast-fails tool calls. Backend catalog limitation, not a Service defect. (Operational note: the deploy endpoint bundles tool+agent creation and 409s on the shared MCP tool, so do #4/#7 via teardown `DELETE /benchmarks/gsm8k/deploy` → redeploy on the default experiment.)

#7 (authbridge sidecar) — partially enactable, and better than kind. `authbridge_enabled=true` turns on the AuthBridge sidecar with the *cluster-default* pipeline; it approximates `full` only if the cluster's Helm base

ships the full pipeline. On ykt2 sessions pass through (3/5 = 0.60) — better than the kind re-run (0.0). The exact preset composition still can't be guaranteed over HTTP.

#5, #6, #8 (plugin presets / observe) — rejected by design. These select a specific per-agent plugin pipeline (layer-3), enacted only by overlaying the `authbridge-config-<agent>` ConfigMap via kubectl. The HTTP-only Service (which here targets a remote cluster with no kubeconfig) returns an actionable **422** rather than silently mis-deploying.

#9, #10 (tau2) — the one regression, and it's environment/infra, not the Service. The tau2 MCP (`exgentic-mcp-tau2`) crash-loops/restarts under any load on ykt2, which stalls MCP calls. Previously `mcp_session._call` had **no per-call timeout** — a single hung session blocked the whole `asyncio.gather` until the run's wall budget fired, so the entire batch reported `failed` with 0 results. This is *worse* than kind (where a clean 3-task once passed); cross-cluster on ykt2 even a 1-task run hung. Not a model or Service-API problem (Qwen3.6-35B tool-calling itself is fine; gsm8k passes). Fix (a) is now **implemented**: `mcp_session._call` bounds every call with `asyncio.timeout` (default 120s, capped by the run budget) *inside* the shared session lock, so a stalled call raises `McpCallTimeout`, fails only its own task, and releases the lock — the batch now preserves the tasks that did pass instead of losing everything. The remaining durable fix (b) is a more stable tau2 MCP image / more node headroom on ykt2, so those tasks stop stalling in the first place. Clear any orphaned `running` runs with `kubectl -n rossoctl-system rollout restart deploy/autobench-service` on ykt3.

#11, #12 (appworld) — blocked upstream by image architecture. The published appworld MCP image is arm64; ykt2 nodes are amd64, so the pod CrashLoopBackOffs with `exec /app/entrypoint.sh: Exec format error` and never serves. (This is a *different* blocker than kind, where the same image instead hangs on its own venv-service health timeout.) Outside what the Service controls either way.

Why the pre-upgrade (2026-08-04, ykt3-only) results looked better

Only **tau2** actually regressed; gsm8k is unchanged within variance and #4/#5/#6/#7/#8 behave the same. Three things changed at once between the 2026-08-04 ykt3-only table and this cross-cluster run, and tau2 is the benchmark most sensitive to all three (multi-turn → a user-simulator LLM driving many MCP round-trips per task):

1. **The MCP moved clusters (dominant factor).** On 2026-08-04 the tau2 MCP + agent ran co-located on ykt3 and were stable enough to finish 17/20. Now the MCP runs on ykt2, where the pod is unstable.
2. **The Rossoctl RC upgrade made the tau2 MCP more crash-prone under load** — independently confirmed on kind (v0.7.0-rc.4): a clean 3-task passed but full-scale 10/20 runs failed at timeout.
3. **The LiteLLM backend changed** (RH `litemaas`); gsm8k barely notices, but tau2's user-simulator makes far more LLM calls per task, so it's more exposed to added latency / rate-limiting.

The latent code weakness (no per-call timeout) was always present but only bites when the MCP is unstable — so it never triggered in the stable 2026-08-04 run.

Operational caveats (for anyone reproducing this)

- **Per-deploy on ykt2 (fix after every `POST /deploy`):** (1) ensure `hf-secret` exists in `team1` (gsm8k MCP env refs `hf-secret/hf-token`; the value can be empty) and `openai-secret` is present; (2)

patch the agent Route `spec.port.targetPort` 8080 → 8000 (rossoctl auto-creates it pointing at the Service port 8080 but endpoints are on 8000 → 503 on the agent card).

- **MLflow reports.** The cross-cluster instance left `mlflow` empty → `/report` returns 409 / 0 records (fail-soft). Pass/fail is authoritative from the run `summary`; `run.json` still exports to S3.
- **Multi-turn needs a big timeout.** tau2 at scale needs `timeout_seconds` well above the 300s default — but note the current tau2 MCP instability means even large timeouts fail here.